

# Student Mental Health Monitoring System

SE LAB Experiment-09 · BATCH-D

## Design Transformation & Implementation Framework

|                  |                                                     |
|------------------|-----------------------------------------------------|
| Submitted By     | Mokshi Shah — 241071068   Vanshika Shah — 241071069 |
| Course / Program | Software Engineering / SY BTech CE                  |
| Institution      | VJTI, Mumbai                                        |
| Date             | April 2026                                          |

### Experiment Objectives

- Optimizing the Object Design Model derived from Lab-07
- Mapping Associations to Collections (DB foreign keys & application-layer arrays)
- Mapping Contracts to Exceptions (pre/post-conditions → HTTP codes & validators)
- Mapping the Object Model to a Persistent Storage Schema (MySQL DDL)
- Implementing Core Modules with traceable code
- Study of Testing Tools applicable to the SMHMS platform

## Table of Contents

|            |                                                           |           |
|------------|-----------------------------------------------------------|-----------|
| <b>1.</b>  | <b>Introduction &amp; Relationship with Previous Labs</b> | <b>3</b>  |
|            | 1.1 Lab-07: Object Model & CRC Cards                      | 3         |
|            | 1.2 Bridge to Lab-09                                      | 3         |
| <b>2.</b>  | <b>Optimization of Object Design Model</b>                | <b>4</b>  |
|            | 2.1 Refinement Strategy                                   | 4         |
|            | 2.2 Design Pattern Integration                            | 4         |
|            | 2.3 Final Optimised Class Summary                         | 5         |
| <b>3.</b>  | <b>Mapping Associations to Collections</b>                | <b>6</b>  |
|            | 3.1 Association-to-Schema Table                           | 6         |
|            | 3.2 Application-Layer Array Representation                | 7         |
| <b>4.</b>  | <b>Mapping Contracts to Exceptions</b>                    | <b>8</b>  |
|            | 4.1 Contract Table                                        | 8         |
|            | 4.2 Sample Enforcement Code                               | 9         |
| <b>5.</b>  | <b>Mapping Object Model to Persistent Storage Schema</b>  | <b>10</b> |
|            | 5.1 Domain Class → MySQL Table Mapping                    | 10        |
|            | 5.2 MySQL DDL Snippets                                    | 11        |
|            | 5.3 Entity-Relationship Overview                          | 13        |
| <b>6.</b>  | <b>Implementation of Core Modules</b>                     | <b>14</b> |
|            | 6.1 Authentication Module                                 | 14        |
|            | 6.2 Screening Module                                      | 14        |
|            | 6.3 Risk Engine Module                                    | 15        |
|            | 6.4 Notification Module                                   | 16        |
|            | 6.5 Counselling Session Module                            | 16        |
| <b>7.</b>  | <b>API Design</b>                                         | <b>17</b> |
| <b>8.</b>  | <b>Study of Testing Tools</b>                             | <b>19</b> |
| <b>9.</b>  | <b>Challenges Faced</b>                                   | <b>20</b> |
| <b>10.</b> | <b>Conclusion</b>                                         | <b>21</b> |

# 1. Introduction & Relationship with Previous Labs

Lab-09 marks the pivotal transition in the software engineering lifecycle where the object-oriented design produced in earlier experiments is translated into a fully operational, production-grade system. For the **Student Mental Health Monitoring System (SMHMS)**, this means converting CRC cards, state machines, inheritance hierarchies, and association tables into running Node.js/Express back-end services, a React front-end, and a MySQL database schema — with every design decision traceable back to a specific artefact from Labs 04–07.

## 1.1 Lab-07: Object Model & CRC Cards

Lab-07 established eleven entity classes (User, ScreeningRecord, MoodLog, RiskProfile, CounselingSession, NotificationLog, PatternReport, ConsentRecord, StudentAcadData, AuditLog, DashboardReport) and four control classes (AuthController, RiskController, NotifController, AnalyticsController). CRC cards specified responsibilities and collaborators; association tables documented 22 named relationships with multiplicities; aggregate diagrams captured composition vs. aggregation boundaries; and state-machine diagrams modelled lifecycles for five key objects.

## 1.2 Bridge to Lab-09

| Lab    | Artefact                         | Lab-09 Manifestation                                  |
|--------|----------------------------------|-------------------------------------------------------|
| Lab-07 | CRC Cards & Associations         | MySQL foreign keys, JOIN queries, JS arrays           |
| Lab-07 | Aggregate diagrams (Composition) | ON DELETE CASCADE constraints                         |
| Lab-07 | State-machine diagrams           | Enum columns + status-guard middleware                |
| Lab-07 | Attribute tables                 | MySQL column types + NOT NULL / UNIQUE                |
| Lab-07 | Inheritance hierarchies          | Single-table inheritance (role column) in users table |

## 2. Optimization of Object Design Model

---

### 2.1 Refinement Strategy

During the transition from the Lab-07 object model to the Lab-09 implementation, three key refinements were applied:

#### Role Centralisation:

The abstract User hierarchy (Student, Counsellor, Admin, Guardian, IT Security Officer) is collapsed into a single `users` table using a discriminator role Enum column. This avoids five-way JOIN queries for authentication and reduces schema complexity without losing any CRC-card responsibilities.

#### Audit Extraction:

In the Lab-07 model `AuditLog` was a collaborator of many classes. In implementation it is extracted as a pure side-effect: every state-modifying route handler fires a blind INSERT into `audit_logs` after committing the primary transaction. Domain classes remain unaware of logging (Single Responsibility Principle).

#### Risk Computation Decoupling:

`RiskController`'s weighted scoring algorithm is extracted into a dedicated `riskEngine.js` service module. Route handlers call the service rather than embedding scoring logic inline — supporting substitutable ML-backed implementations in future iterations.

### 2.2 Design Pattern Integration

#### Observer — Audit Logging

Every route handler that mutates state (submit screening, book session, toggle consent) executes an unconditional INSERT into `audit_logs` after the primary DB write. The route handler is the subject; audit insertion is the observer action. No domain class participates in logging.

#### Strategy — Risk Scoring

`riskEngine.js` accepts a `RiskInputDTO` (PHQ-9, GAD-7, PSS, mood trend, academic engagement) and applies a configurable weight vector to produce a composite score. The weight vector is loaded from `SystemConfig` at startup, so administrators can substitute a different scoring strategy without code changes.

#### Facade — REST API Layer

Express route files present a clean REST surface to the React front-end, hiding multi-table JOIN queries, encryption/decryption of session notes, and ML model calls. The front-end components are entirely unaware of the database schema.

#### Template Method — Screening Lifecycle

The screening submission workflow follows a fixed sequence: (1) Validate token → (2) Persist partial/complete record → (3) Invoke `riskEngine` → (4) Update `RiskProfile` → (5) Trigger notification if High → (6) Write audit entry. Each step is a fixed hook; the scoring and notification logic are the variable parts.

#### State Guard Middleware

Status transitions enforced by Lab-07 state machines (e.g., `CounselingSession` may only move Requested → Confirmed, not Completed → Confirmed) are implemented as Express middleware functions that validate current status before any PUT operation.

2.3 Final Optimised Class Summary

| Class             | Lab-07 Change                                          | Rationale                                                          |
|-------------------|--------------------------------------------------------|--------------------------------------------------------------------|
| User (abstract)   | Merged into single users table with role discriminator | Eliminates 5-table UNION; preserves is-a semantics via role column |
| ScreeningRecord   | No change; maps directly to screening_records          | Attributes align 1-to-1 with MySQL columns                         |
| MoodLog           | logDate unique-per-student constraint added            | Enforces Lab-07 rule: one log per day per student                  |
| RiskProfile       | alertSent reset via trigger on score drop              | Prevents stale High-risk suppression after recovery                |
| CounselingSession | sessionNotes AES-256 at application layer              | Meets privacy-by-design requirement from Lab-07                    |
| NotificationLog   | Retry counter column added                             | Supports Lab-07 retry-on-failure responsibility                    |
| PatternReport     | FK to ml_model_versions table added                    | Traceability for model version used per report                     |
| ConsentRecord     | Soft-delete via revokedAt timestamp                    | Preserves audit trail on revocation                                |
| StudentAcadData   | Partitioned by academic year                           | Performance optimisation for large cohorts                         |
| AuditLog          | hashChain column enforced by DB trigger                | Tamper-evidence per Lab-07 HMAC requirement                        |
| DashboardReport   | Subtype discrimination via report_type Enum            | Single-table inheritance mirrors Lab-07 hierarchy                  |

### 3. Mapping Associations to Collections

Every association identified in Lab-07 is realised either as a relational foreign key in MySQL or as a JavaScript array/object at the API and UI layers. The table below documents each mapping.

#### 3.1 Association-to-Schema Table

| Lab-07 ID | Class A         | Rel.                 | Class B           | DB Foreign Key                                           | API Representation                          |
|-----------|-----------------|----------------------|-------------------|----------------------------------------------------------|---------------------------------------------|
| A01       | Student         | submits 0..*         | ScreeningRecord   | screening_records.student_id → users.id                  | GET /api/student/screenings → Array         |
| A02       | Student         | creates 0..*         | MoodLog           | mood_logs.student_id → users.id                          | GET /api/student/moodlogs → Array           |
| A03       | Student         | owns 1               | RiskProfile       | risk_profiles.student_id UNIQUE → users.id               | GET /api/student/risk → Single object       |
| A04       | Student         | requests 0..*        | CounselingSession | counseling_sessions.student_id → users.id                | GET /api/student/sessions → Array           |
| A05       | Student         | grants 0..1          | ConsentRecord     | consent_records.student_id UNIQUE → users.id             | GET /api/student/consent → Single object    |
| A06       | Counsellor      | manages 0..*         | CounselingSession | counseling_sessions.counsellor_id → users.id             | GET /api/counsellor/sessions → Array        |
| A07       | Counsellor      | reviews 0..*         | RiskProfile       | No FK; filtered query by assigned counsellor             | GET /api/counsellor/atrisk → Array          |
| A08       | Admin           | manages 0..*         | User              | No FK; admin role in session                             | GET /api/admin/users → Array                |
| A12       | RiskProfile     | derived from 1..*    | ScreeningRecord   | risk_profiles.latest_screening_id → screening_records.id | Embedded JSON in RiskProfile response       |
| A13       | RiskProfile     | derived from 1..*    | PatternReport     | risk_profiles.latest_pattern_id → pattern_reports.id     | Embedded JSON in RiskProfile response       |
| A15       | PatternReport   | produced from 1..*   | StudentAcadData   | student_acad_data.student_id → users.id                  | Internal service call; not exposed directly |
| A16       | NotificationLog | sent to 1            | User              | notification_logs.recipient_id → users.id                | GET /api/notifications → Array              |
| A19       | AuditLog        | records actions of 1 | User              | audit_logs.actor_id → users.id (nullable on anon)        | GET /api/admin/auditlogs → Array            |
| A21       | DashboardReport | aggregates 0..*      | RiskProfile       | No FK; aggregate query at generation time                | GET /api/admin/dashboard → Aggregated JSON  |

#### 3.2 Application-Layer Array Representation

The following code snippet illustrates the Counsellor at-risk queue — an example of a Lab-07 A07 association (Counsellor reviews RiskProfile) being realised as a JavaScript array via a multi-table JOIN:

```
// riskRoutes.js – GET /api/counsellor/atrisk
router.get('/atrisk', requireRole('counsellor'), async (req, res) => {
  try {
    const [rows] = await pool.query(`
```

```
SELECT u.id, u.name, u.email,
rp.composite_score, rp.risk_tier, rp.classified_at,
rp.alert_sent
FROM risk_profiles rp
JOIN users u ON rp.student_id = u.id
WHERE rp.risk_tier IN ('Medium', 'High')
ORDER BY rp.composite_score DESC`,
);
res.json(rows); // rows is a JS Array – association made concrete
} catch (err) {
res.status(500).json({ message: 'Error fetching at-risk list' });
}
});
```

## 4. Mapping Contracts to Exceptions

Lab-07 state machines and CRC responsibilities imply formal contracts (pre-conditions, post-conditions, invariants). In the implementation these are enforced through HTTP status codes, middleware guards, MySQL constraints, and try/catch blocks.

### 4.1 Contract Table

| Contract                                                         | Type           | Mechanism                                                                                  | HTTP Response                   |
|------------------------------------------------------------------|----------------|--------------------------------------------------------------------------------------------|---------------------------------|
| User must be authenticated to access any /api/* route            | Pre-condition  | Passport.js session check; req.user non-null                                               | 401 Unauthorised                |
| Student may submit at most one MoodLog per calendar day          | Invariant      | UNIQUE KEY (student_id, log_date) on mood_logs table                                       | 409 Conflict (DB duplicate key) |
| PHQ-9 score must be 0–27, GAD-7 0–21, PSS 0–40                   | Pre-condition  | Joi schema validation middleware before INSERT                                             | 422 Unprocessable Entity        |
| ScreeningRecord must be in Submitted state before scoring        | Pre-condition  | Status guard middleware checks is_complete = true                                          | 409 Conflict                    |
| CounselingSession may only be Confirmed from Requested state     | Pre-condition  | State-guard middleware validates current status                                            | 409 Conflict                    |
| After screening submission, RiskProfile must be updated          | Post-condition | Transactional: screening INSERT + risk_profiles UPSERT in one transaction                  | 500 on rollback                 |
| High-risk alert must be sent when riskTier transitions to High   | Post-condition | riskEngine.js emits alert event; NotifController listener fires INSERT                     | Logged asynchronously           |
| ConsentRecord must exist before Guardian can access student data | Pre-condition  | consentGuard middleware queries consent_records                                            | 403 Forbidden                   |
| AuditLog entries must be tamper-evident                          | Invariant      | BEFORE INSERT trigger computes HMAC(prev_hash, new_row) → hash_chain                       | Cannot be bypassed at app layer |
| Session notes are counsellor-only                                | Invariant      | Role check in route + AES-256 key held only by counsellor service                          | 403 Forbidden                   |
| PatternReport requires ≥4 weeks of data                          | Pre-condition  | data_sufficient flag evaluated before ML call; returns 200 with flag=false if insufficient | 200 with dataSufficient: false  |
| Notification frequency cap: max 3 per student per day            | Invariant      | NotifController queries COUNT from notification_logs WHERE DATE = today                    | Silent drop; logged in audit    |

### 4.2 Sample Enforcement Code

The snippet below demonstrates the PHQ-9/GAD-7/PSS pre-condition and the post-condition that RiskProfile must be updated atomically with the screening record:

```
// screeningRoutes.js – POST /api/student/screenings
const Joi = require('joi');

const screeningSchema = Joi.object({
  phq9Score: Joi.number().integer().min(0).max(27).required(),
```



```
gad7Score: Joi.number().integer().min(0).max(21).required(),
pssScore: Joi.number().integer().min(0).max(40).required(),
});

router.post('/', requireRole('student'), async (req, res) => {
  const { error, value } = screeningSchema.validate(req.body);
  if (error) // Pre-condition: score ranges
    return res.status(422).json({ message: error.details[0].message });

  const conn = await pool.getConnection();
  try {
    await conn.beginTransaction();
    const [rec] = await conn.query(
      `INSERT INTO screening_records
      (student_id, phq9_score, gad7_score, pss_score, is_complete)
      VALUES (?, ?, ?, ?, 1)`,
      [req.user.id, value.phq9Score, value.gad7Score, value.pssScore]
    );
    // Post-condition: risk profile updated in same transaction
    const { score, tier } = riskEngine.score(value);
    await conn.query(
      `INSERT INTO risk_profiles (student_id, composite_score, risk_tier,
      classified_at, alert_sent)
      VALUES (?, ?, ?, NOW(), 0)
      ON DUPLICATE KEY UPDATE
      composite_score=VALUES(composite_score),
      risk_tier=VALUES(risk_tier),
      classified_at=NOW(), alert_sent=0`,
      [req.user.id, score, tier]
    );
    await conn.commit();
    if (tier === 'High') notifController.sendAlert(req.user.id);
    res.status(201).json({ recordId: rec.insertId, riskTier: tier });
  } catch (err) {
    await conn.rollback();
    res.status(500).json({ message: 'Screening submission failed' });
  } finally { conn.release(); }
});
```

## 5. Mapping Object Model to Persistent Storage Schema

### 5.1 Domain Class → MySQL Table Mapping

| Domain Class (Lab-07)          | MySQL Table         | PK             | Key FKs                                                | Notes                                       |
|--------------------------------|---------------------|----------------|--------------------------------------------------------|---------------------------------------------|
| User (abstract) — all roles    | users               | id UUID        | —                                                      | role ENUM discriminator; twofa_enabled BOOL |
| ScreeningRecord                | screening_records   | id UUID        | student_id → users.id                                  | phq9, gad7, pss scores; risk_tier ENUM      |
| MoodLog                        | mood_logs           | id UUID        | student_id → users.id                                  | UNIQUE(student_id, log_date)                |
| RiskProfile                    | risk_profiles       | id UUID        | student_id UNIQUE → users.id; counsellor_id → users.id | composite_score FLOAT; trend_data JSON      |
| CounselingSession              | counseling_sessions | id UUID        | student_id → users.id; counsellor_id → users.id        | status ENUM; session_notes encrypted TEXT   |
| NotificationLog                | notification_logs   | id UUID        | recipient_id → users.id                                | retry_count INT; ack_at DATETIME            |
| PatternReport                  | pattern_reports     | id UUID        | student_id → users.id                                  | anomalies JSON; data_sufficient BOOL        |
| ConsentRecord                  | consent_records     | id UUID        | student_id UNIQUE → users.id; guardian_id → users.id   | revoked_at nullable                         |
| StudentAcadData                | student_acad_data   | id UUID        | student_id → users.id                                  | grades JSON; engagement_score FLOAT         |
| AuditLog                       | audit_logs          | id BIGINT AUTO | actor_id → users.id (nullable)                         | hash_chain VARCHAR; no cascade deletes      |
| DashboardReport (all subtypes) | dashboard_reports   | id UUID        | —                                                      | report_type ENUM; scope JSON; format ENUM   |

### 5.2 MySQL DDL Snippets

#### users (collapses User hierarchy)

```
CREATE TABLE users (
  id CHAR(36) NOT NULL DEFAULT (UUID()),
  email VARCHAR(191) NOT NULL,
  password_hash VARCHAR(255) NOT NULL,
  role ENUM('Student','Counsellor','Admin',
    'Guardian','ITOfficer') NOT NULL,
  locked TINYINT(1) NOT NULL DEFAULT 0,
  failed_attempts TINYINT NOT NULL DEFAULT 0,
  twofa_enabled TINYINT(1) NOT NULL DEFAULT 0,
  created_at DATETIME NOT NULL DEFAULT NOW(),
```

```
last_login_at DATETIME,  
PRIMARY KEY (id),  
UNIQUE KEY uq_email (email)  
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;
```

### screening\_records

```
CREATE TABLE screening_records (  
  id CHAR(36) NOT NULL DEFAULT (UUID()),  
  student_id CHAR(36) NOT NULL,  
  phq9_score TINYINT NOT NULL CHECK (phq9_score BETWEEN 0 AND 27),  
  gad7_score TINYINT NOT NULL CHECK (gad7_score BETWEEN 0 AND 21),  
  pss_score TINYINT NOT NULL CHECK (pss_score BETWEEN 0 AND 40),  
  risk_tier ENUM('Low','Medium','High') NOT NULL,  
  version SMALLINT NOT NULL DEFAULT 1,  
  is_complete TINYINT(1) NOT NULL DEFAULT 0,  
  submitted_at DATETIME NOT NULL DEFAULT NOW(),  
  PRIMARY KEY (id),  
  CONSTRAINT fk_sr_student FOREIGN KEY (student_id)  
  REFERENCES users(id) ON DELETE CASCADE  
) ENGINE=InnoDB;
```

### mood\_logs — UNIQUE constraint enforcing one-per-day

```
CREATE TABLE mood_logs (  
  id CHAR(36) NOT NULL DEFAULT (UUID()),  
  student_id CHAR(36) NOT NULL,  
  mood_score TINYINT NOT NULL CHECK (mood_score BETWEEN 1 AND 10),  
  sleep_hours FLOAT NOT NULL CHECK (sleep_hours BETWEEN 0 AND 24),  
  stressors VARCHAR(500),  
  log_date DATE NOT NULL,  
  is_partial TINYINT(1) NOT NULL DEFAULT 0,  
  PRIMARY KEY (id),  
  UNIQUE KEY uq_one_per_day (student_id, log_date),  
  CONSTRAINT fk_ml_student FOREIGN KEY (student_id)  
  REFERENCES users(id) ON DELETE CASCADE  
) ENGINE=InnoDB;
```

### risk\_profiles

```
CREATE TABLE risk_profiles (  
  id CHAR(36) NOT NULL DEFAULT (UUID()),  
  student_id CHAR(36) NOT NULL,
```

```
composite_score FLOAT NOT NULL DEFAULT 0.0,
risk_tier ENUM('Low','Medium','High') NOT NULL DEFAULT 'Low',
trend_data JSON,
classified_at DATETIME NOT NULL DEFAULT NOW(),
alert_sent TINYINT(1) NOT NULL DEFAULT 0,
counsellor_id CHAR(36),
PRIMARY KEY (id),
UNIQUE KEY uq_rp_student (student_id),
CONSTRAINT fk_rp_student FOREIGN KEY (student_id)
REFERENCES users(id) ON DELETE CASCADE,
CONSTRAINT fk_rp_counsellor FOREIGN KEY (counsellor_id)
REFERENCES users(id) ON DELETE SET NULL
) ENGINE=InnoDB;
```

#### audit\_logs with tamper-evident hash chain trigger

```
CREATE TABLE audit_logs (
id BIGINT UNSIGNED NOT NULL AUTO_INCREMENT,
actor_id CHAR(36),
action_type ENUM('Login','DataAccess','ConfigChange',
'AlertAck','Export','ConsentChange') NOT NULL,
target_type VARCHAR(64) NOT NULL,
target_id CHAR(36) NOT NULL,
ip_address VARCHAR(45),
hash_chain VARCHAR(64) NOT NULL DEFAULT '',
ts DATETIME NOT NULL DEFAULT NOW(),
PRIMARY KEY (id),
CONSTRAINT fk_al_actor FOREIGN KEY (actor_id)
REFERENCES users(id) ON DELETE SET NULL
) ENGINE=InnoDB;

-- Tamper-evidence trigger
DELIMITER $$

CREATE TRIGGER trg_audit_hash BEFORE INSERT ON audit_logs FOR EACH ROW
BEGIN
DECLARE prev_hash VARCHAR(64) DEFAULT '0';
SELECT hash_chain INTO prev_hash FROM audit_logs ORDER BY id DESC LIMIT 1;
SET NEW.hash_chain = SHA2(CONCAT(prev_hash, NEW.actor_id,
NEW.action_type, NEW.ts), 256);
END$$

DELIMITER ;
```

### 5.3 Entity-Relationship Overview

The central entity is **users**. All student-owned data (`screening_records`, `mood_logs`, `risk_profiles`, `counseling_sessions`, `consent_records`, `student_acad_data`, `notification_logs`) references it via `student_id` with ON DELETE CASCADE — consistent with the Lab-07 Composition aggregates. The `audit_logs` table uses ON DELETE SET NULL to preserve forensic history even after account deletion. The `risk_profiles` table references `screening_records` and `pattern_reports` via nullable FK columns (Lab-07 Aggregation, not Composition).

## 6. Implementation of Core Modules

### 6.1 Authentication Module

Authentication is implemented using **Passport.js** with the passport-local strategy (email + bcrypt password) and an optional passport-totp strategy for 2FA. On successful login, the resolved user object (id, email, role) is serialised into an Express session with a 24-hour TTL. The locked flag is checked before password comparison; five consecutive failures set locked = 1 and return 423 Locked. All login events (success, failure, lockout) emit an Observer-pattern audit INSERT.

```
// authRoutes.js – POST /api/auth/login
router.post('/login', async (req, res) => {
  const { email, password } = req.body;
  const [[user]] = await pool.query(
    'SELECT * FROM users WHERE email = ?', [email]);
  if (!user) return res.status(401).json({ message: 'Invalid credentials' });
  if (user.locked) return res.status(423).json({ message: 'Account locked' });
  const match = await bcrypt.compare(password, user.password_hash);
  if (!match) {
    await pool.query(
      'UPDATE users SET failed_attempts = failed_attempts + 1,
        locked = (failed_attempts + 1 >= 5) WHERE id = ?', [user.id]);
    return res.status(401).json({ message: 'Invalid credentials' });
  }
  await pool.query(
    'UPDATE users SET failed_attempts=0, last_login_at=NOW() WHERE id=?',
    [user.id]);
  req.session.user = { id: user.id, role: user.role, email: user.email };
  auditLog(user.id, 'Login', 'users', user.id, req.ip);
  res.json({ role: user.role });
});
```

### 6.2 Screening Module

The screening module exposes POST /api/student/screenings for submission and GET /api/student/screenings for history retrieval. Joi validates PHQ-9 (0–27), GAD-7 (0–21), and PSS (0–40) ranges before any DB write. The submission route opens a MySQL transaction, inserts the screening record, invokes riskEngine.js, UPSERTs the risk\_profiles row, commits, and — if the computed tier is High — triggers a counsellor alert notification asynchronously. Partial saves set is\_complete = 0 and skip risk computation.

### 6.3 Risk Engine Module

riskEngine.js implements the Strategy pattern. The default strategy applies the following configurable weights:

| Input              | Weight | Score Range | Normalised Contribution |
|--------------------|--------|-------------|-------------------------|
| PHQ-9 (depression) | 35 %   | 0–27        | 0–35 pts                |

| Input               | Weight | Score Range       | Normalised Contribution |
|---------------------|--------|-------------------|-------------------------|
| GAD-7 (anxiety)     | 25 %   | 0–21              | 0–25 pts                |
| PSS (stress)        | 20 %   | 0–40              | 0–20 pts                |
| Mood trend (7-day)  | 10 %   | 1–10              | 0–10 pts                |
| Academic engagement | 10 %   | 0–1<br>normalised | 0–10 pts                |

Tier thresholds: Low = 0–39, Medium = 40–69, High  $\geq$  70. The weight vector is loaded from the `system_config` table at server startup. Reloading the config row and restarting the weights object substitutes the scoring strategy without a code deploy.

```
// riskEngine.js

let W = { phq9: 0.35, gad7: 0.25, pss: 0.20, mood: 0.10, acad: 0.10 };

exports.score = function({ phq9Score, gad7Score, pssScore,
moodTrend = 5, acadScore = 0.5 }) {
  const composite =
    (phq9Score / 27) * W.phq9 * 100 +
    (gad7Score / 21) * W.gad7 * 100 +
    (pssScore / 40) * W.pss * 100 +
    (moodTrend / 10) * W.mood * 100 +
    (1 - acadScore) * W.acad * 100;
  const tier = composite >= 70 ? 'High'
: composite >= 40 ? 'Medium' : 'Low';
  return { score: parseFloat(composite.toFixed(2)), tier };
};

exports.reloadWeights = async function(pool) {
  const [[cfg]] = await pool.query(
    'SELECT config_value FROM system_config WHERE config_key = ?',
    ['risk_weights']);
  if (cfg) W = JSON.parse(cfg.config_value);
};
```

## 6.4 Notification Module

NotifController implements the Observer pattern. All route handlers that produce state changes call `notifController.emit(type, payload)`. The controller looks up the recipient's preferred channel, enforces the frequency cap (max 3 per student per 24 h), composes the subtype-specific message, attempts email delivery via Nodemailer, and inserts a `notification_logs` row recording delivery status. On email bounce a retry is scheduled (max 3 retries); on exhaustion the notification is demoted to in-app only.

## 6.5 Counselling Session Module

Session state transitions are guarded by the `requireStatus` middleware. `PUT /api/counsellor/sessions/:id/confirm` first queries the current status; if it is not 'Requested' a 409 is returned. Confirmed sessions trigger a 24-hour reminder via

the NotifController. Session notes are AES-256-CBC encrypted using a key derived from the counsellor's userId + a server-side secret; the ciphertext is stored in session\_notes and decrypted only when the counsellor's authenticated session requests the record.



## 7. API Design

The SMHMS backend exposes a resource-oriented REST API. All routes are role-prefixed (/api/student/, /api/counsellor/, /api/admin/) to make RBAC transparent. The table below documents the complete endpoint catalogue.

| Endpoint                              | Method | Description                         | DB Operation                                               |
|---------------------------------------|--------|-------------------------------------|------------------------------------------------------------|
| /api/auth/login                       | POST   | Authenticate; return session        | SELECT users; UPDATE failed_attempts                       |
| /api/auth/logout                      | POST   | Destroy session                     | None                                                       |
| /api/auth/me                          | GET    | Return session user object          | None (session read)                                        |
| /api/student/screenings               | POST   | Submit screening (validated)        | INSERT screening_records + UPSERT risk_profiles            |
| /api/student/screenings               | GET    | Retrieve history                    | SELECT screening_records WHERE student_id                  |
| /api/student/moodlogs                 | POST   | Submit daily mood log               | INSERT mood_logs (UNIQUE guard)                            |
| /api/student/moodlogs                 | GET    | Retrieve mood history               | SELECT mood_logs WHERE student_id                          |
| /api/student/risk                     | GET    | Get own risk profile                | SELECT risk_profiles WHERE student_id                      |
| /api/student/sessions                 | POST   | Request counselling session         | INSERT counseling_sessions                                 |
| /api/student/sessions                 | GET    | List own sessions                   | SELECT counseling_sessions WHERE student_id                |
| /api/student/consent                  | PUT    | Grant/revoke guardian consent       | UPSERT consent_records                                     |
| /api/student/notifications            | GET    | List notifications                  | SELECT notification_logs WHERE recipient_id                |
| /api/counsellor/atrisk                | GET    | Get at-risk student list            | SELECT risk_profiles JOIN users WHERE tier IN Medium/High  |
| /api/counsellor/sessions              | GET    | Get assigned sessions               | SELECT counseling_sessions WHERE counsellor_id             |
| /api/counsellor/sessions/:id/confirm  | PUT    | Confirm session (state guard)       | UPDATE status; INSERT notification_logs                    |
| /api/counsellor/sessions/:id/notes    | PUT    | Add encrypted session notes         | UPDATE session_notes (AES-256)                             |
| /api/counsellor/sessions/:id/complete | PUT    | Mark session complete               | UPDATE status + audit INSERT                               |
| /api/admin/users                      | GET    | List all users                      | SELECT users                                               |
| /api/admin/users/:id/lock             | PUT    | Lock/unlock user account            | UPDATE users.locked + audit                                |
| /api/admin/config                     | PUT    | Update system config / risk weights | UPSERT system_config + riskEngine.reloadWeights()          |
| /api/admin/dashboard                  | GET    | Generate anonymised KPI report      | Aggregate SELECT across risk_profiles, counseling_sessions |
| /api/admin/auditlogs                  | GET    | Retrieve audit trail                | SELECT audit_logs ORDER BY ts DESC                         |
| /api/analytics/patterns/:studentId    | POST   | Trigger pattern analysis            | INSERT pattern_reports; UPSERT risk_profiles               |

## 8. Study of Testing Tools

A comprehensive testing strategy is required before the Lab-10 test plan. The categories below are directly mapped to SMHMS components.

### Automation (E2E)

| Tool               | Type               | SMHMS Application                                                                   |
|--------------------|--------------------|-------------------------------------------------------------------------------------|
| Cypress            | E2E + Component    | Test student screening form; counsellor confirm flow; admin dashboard KPI rendering |
| Playwright         | Cross-browser E2E  | Verify mood log UNIQUE-key error shown correctly in all browsers                    |
| Selenium WebDriver | Browser automation | Legacy IE/Edge compatibility for admin portal                                       |

### API Testing

| Tool             | Capability              | SMHMS Application                                                                       |
|------------------|-------------------------|-----------------------------------------------------------------------------------------|
| Postman / Newman | REST collection runner  | 24 endpoint regression suite; environment variables for student/counsellor/admin tokens |
| Supertest (Jest) | In-process HTTP testing | Unit-test route handlers without a live server; mock riskEngine                         |
| Pact             | Contract testing        | Verify React front-end contract matches Express API response shapes                     |

### Performance Testing

| Tool          | Focus             | SMHMS Use Case                                                     |
|---------------|-------------------|--------------------------------------------------------------------|
| Apache JMeter | Load / stress     | Simulate 500 concurrent screening submissions; measure P95 latency |
| k6            | Script-based load | Ramp-up test for notification burst during semester start          |

### Security Testing

| Tool             | Focus               | SMHMS Relevance                                                          |
|------------------|---------------------|--------------------------------------------------------------------------|
| OWASP ZAP        | DAST                | Scan /api/* for SQL injection in score fields; session fixation on login |
| Burp Suite       | Pen testing         | Verify counsellor session notes endpoint refuses non-counsellor JWTs     |
| npm audit / Snyk | Dependency CVE scan | Continuous scan of Express, mysql2, bcrypt dependencies                  |

## 9. Challenges Faced

---

### Collapsing the User Hierarchy

The Lab-07 five-subclass User hierarchy needed to map to a single users table without losing role-specific attributes. The solution was a role ENUM discriminator column combined with a role\_metadata JSON column holding sparse role-specific fields (e.g., licence\_no for Counsellor, clearance\_level for ITOfficer). This avoids five-table JOINS while preserving all CRC-card attributes.

### Transactional Risk Update

Ensuring the post-condition 'RiskProfile must be updated atomically with ScreeningRecord' required wrapping two INSERTs in a MySQL transaction with proper rollback on the risk engine failure. Early implementations missed the connection.release() in the finally block, causing pool exhaustion under load — resolved by strict try/catch/finally discipline.

### Tamper-Evident Audit Log

Implementing the HMAC hash chain as a MySQL BEFORE INSERT trigger required the trigger to read the previous row's hash\_chain under a gap-lock. Under concurrent inserts this caused deadlocks. The solution was to serialise audit inserts through a dedicated Node.js queue (p-queue, concurrency=1) rather than relying on the trigger alone for ordering.

### State-Guard Middleware Reuse

Every stateful entity (CounselingSession, ScreeningRecord, NotificationLog) required its own state-transition guards. Writing six separate middleware functions duplicated logic. The solution was a generic requireStatus(table, allowedStates) factory function that queries the current status of any record by its :id parameter before passing control to the route handler.

### Session Note Encryption Key Management

AES-256-CBC encryption of counselling session notes required a per-counsellor key that is not stored server-side in plaintext. The chosen approach derives the encryption key using PBKDF2 from the counsellor's hashed userId + a server secret, avoiding a separate key store while maintaining that only the authorised counsellor's authenticated session can decrypt the notes.

### Notification Frequency Cap Under Concurrency

The cap of 3 notifications per student per 24 hours required an atomic read-then-insert to avoid race conditions. A naive SELECT COUNT + INSERT pattern allowed bursts under concurrency. The fix was an INSERT ... WHERE (SELECT COUNT...) < 3 sub-select combined with a UNIQUE constraint on (recipient\_id, DATE(created\_at), notification\_type) to make over-delivery a DB-level impossibility.

## 10. Conclusion

---

Lab-09 successfully demonstrates the full lifecycle of model-driven software engineering for the Student Mental Health Monitoring System. Beginning with CRC cards and state machines in Lab-07, every design decision has been traced forward to a concrete implementation artefact in this experiment.

The eleven Lab-07 entity classes map one-to-one to MySQL tables. The twenty-two named associations map to FK constraints and JOIN queries. The five state machines map to ENUM columns and status-guard middleware. The three inheritance hierarchies map to single-table discriminated models. The Lab-07 CRC-card collaborator lists map to Express service calls and module imports. No design decision was discarded; each was either preserved directly or refined with an explicit engineering rationale.

The four design patterns integrated in this experiment — Observer (audit logging), Strategy (risk scoring), Facade (REST API), and Template Method (screening lifecycle) — each have direct, line-traceable manifestations in the codebase. The privacy-by-design requirements from Lab-07 (AES-256 session notes, HMAC audit chain, consent gate for guardian access) are enforced at the database, application, and transport layers.

The most important outcome of this experiment is the confirmation that the software engineering process — from requirements analysis through object modelling to implementation — is not ceremonial overhead. The structured Lab-07 design reduced implementation ambiguity, surfaced constraint conflicts before code was written, and provides the traceability matrix that Lab-10 test planning will build upon directly.